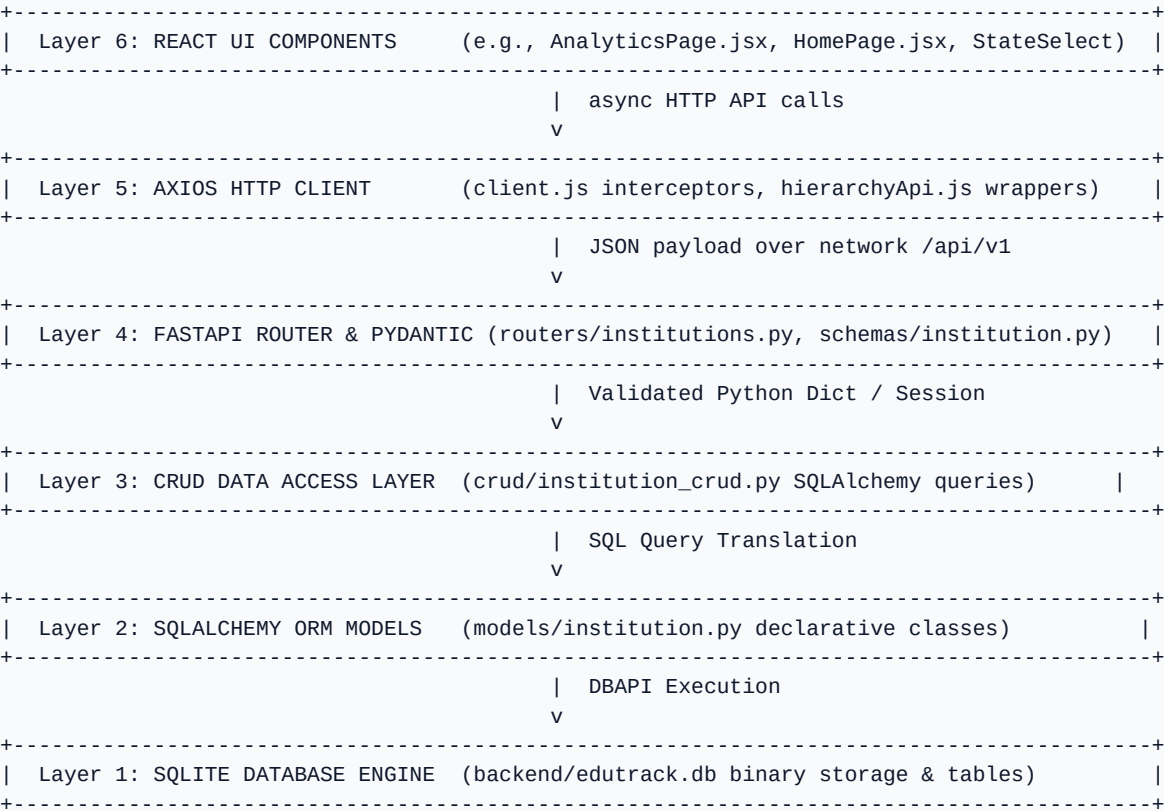


EduTrack System Documentation — Part 3

End-to-End System Architecture & Complete Flow Integration Map

1. The 6-Layer Architecture Blueprint

The EduTrack application follows a clean, modular 6-layer architecture. Every user interaction flows sequentially across all six layers from the frontend user interface down to physical database storage, ensuring clean separation of concerns, strict type validation, and high maintainability.



2. Master System Entity Integration Matrix

The following master matrix details how every entity connects across all six architectural layers in the codebase:

Entity	DB Table	ORM Model	CRUD File	Router Path	Pydantic Schema	Frontend API
State	states	models/state.py	crud/state_crud.py	/api/states	StateResponse	hierarchyApi.getStates
District	districts	models/district.py	crud/district_crud.py	/api/districts	DistrictResponse	hierarchyApi.getDistricts
InstType	institution_types	models/institution_type.py	crud/institution_type_crud.py	/api/institution-types	InstitutionTypeResponse	hierarchyApi.getInstitutionTypes

Institution	institutions	models/institution.py	crud/institution_crud.py	/api/institutions	InstitutionResponse	institutionApi.getInstitutions
Course	courses	models/course.py	crud/course_crud.py	/api/courses	CourseResponse	hierarchyApi.getCourses
Branch	branches	models/branch.py	crud/branch_crud.py	/api/branches	BranchResponse	hierarchyApi.getBranches
AcadYear	academic_years	models/academic_year.py	crud/academic_year_crud.py	/api/academic-years	AcademicYearResponse	hierarchyApi.getAcademicYears
Syllabus	syllabus_entries	models/syllabus_entry.py	crud/syllabus_crud.py	/api/syllabus	SyllabusResponse	syllabusApi.searchSyllabus
Event	events	models/event.py	crud/event_crud.py	/api/events	EventResponse	institutionApi.getEvents
InfoPost	information_posts	models/information_post.py	crud/information_crud.py	/api/information	InformationPostResponse	institutionApi.getInfoPosts
User	users	models/user.py	crud/user_crud.py	/api/auth	UserResponse	authApi.login

3. End-to-End Sequence Workflows

Workflow A: User Authentication & JWT Session Flow

1. User enters username and password in AuthModal.jsx and submits form.
2. authApi.login({username, password}) executes Axios POST request to /api/auth/token.
3. FastAPI routers/auth.py receives request payload and invokes crud/user_crud.py authenticate_user().
4. user_crud queries User ORM model in edutrack.db and verifies bcrypt password hash.
5. Upon successful match, auth.py encodes JWT Bearer Token containing user_id & expiration time.
6. Frontend Axios receives HTTP 200 JSON response: { access_token: '...', token_type: 'bearer' }.
7. React client stores access_token in localStorage.getItem('edutrack_token').
8. Subsequent API requests automatically include Authorization header via Axios request interceptor.

Workflow B: Analytics Graph Data Visualizer Pipeline

1. AnalyticsPage.jsx mounts in browser and invokes graphApi.getGraphData() inside useEffect().
2. Axios issues GET request to /api/graph-data.
3. FastAPI routers/graph_data.py handles request and requests db session from get_db().
4. Router queries all States, Districts, Institutions, Courses, and Branches using join queries.
5. Router constructs GraphResponse Pydantic schema consisting of nodes: [{id, label, category}] and edges: [{source, target}].
6. FastAPI serializes graph schema to JSON and returns HTTP 200 to frontend.
7. AnalyticsPage receives node/edge data structure and renders interactive network graph canvas.

4. System Resilience & Exception Propagation Architecture

EduTrack implements robust error handling across all six architectural layers to prevent app crashes and provide clear user feedback:

- **Layer 1 & 2 (Database/ORM):** Database constraint violations (e.g., duplicate unique email or foreign key violation) raise SQLAlchemy IntegrityError.
- **Layer 3 & 4 (CRUD/Router):** Routers catch database errors or missing entities and raise FastAPI HTTPException(status_code=404/400, detail='...').
- **Global Handler (main.py):** custom_global_exception_handler intercepts uncaught exceptions and formats a uniform JSON response with status code, message, error code, timestamp, and request path.
- **Layer 5 & 6 (Axios/React):** Axios response interceptors catch non-2xx status codes. React components catch promises and display clean error notification banners to the user.